Machine Learning Project Documentation

Assignment 3

Model Refinement, Test Submission & Deployment

PART A: Model Refinement

1. Overview

This section focuses on the model refinement phase of the machine learning project. The main objective is to improve the prediction of school electricity access using socio-economic, infrastructure, and satellite-based indicators. The target variable is Electricity_Access_Percent, while the input features include GDP per capita, national electricity access, urban population, education expenditure, and NASA light-based variables such as Lpc, stbSoL, Pdens, and wGini_L050.

In the previous model comparison stage, Random Forest, XGBoost, and Gradient Boosting were evaluated, and XGBoost was selected as the best-performing algorithm. In this updated Part A notebook, the refinement phase was improved methodologically by using a group-based train-test split and group-based cross-validation. Since the dataset contains multiple yearly observations for the same country, standard random splitting could allow observations from the same country to appear in both the training and test sets. To reduce this risk, GroupShuffleSplit and GroupKFold were used with ISO country codes as groups.

The refinement process includes three main steps: evaluating the initial XGBoost model, applying GridSearchCV for hyperparameter tuning, and using feature importance-based feature selection. The final model is selected primarily according to cross-validation performance because cross-validation provides a more reliable estimate than a single test split.

2. Model Evaluation

The initial XGBoost model was trained using 100 estimators, a learning rate of 0.1, and a maximum depth of 4. The model achieved strong performance on the held-out test set, with an R2 score of 0.9209, an MAE of 1.6716, and an RMSE of 5.8279. These results show that the model explained a high proportion of the variation in school electricity access and produced relatively low prediction errors.

However, the cross-validation results showed that the model performance was not equally stable across all country-based folds. The initial XGBoost model achieved a mean cross-validation R2 score of 0.7008 with a standard deviation of 0.1837. This indicated that although the single test-set result was strong, the model could still benefit from refinement to improve stability and generalization across different country groups.

```
model_comparison = pd.DataFrame({
    "Model": [
        "Random Forest",
        "XGBoost",
        "Gradient Boosting"
    ],
    "R2 Score": [
        rf_r2,
        xgb_r2,
        gb_r2
    ],
    "MAE": [
        rf_mae,
        xgb_mae,
        gb_mae
    ],
    "RMSE": [
        rf_rmse,
        xgb_rmse,
        gb_rmse
    ]
})

model_comparison
```

	Model	R2 Score	MAE	RMSE
0	Random Forest	0.904563	2.702273	6.401391
1	XGBoost	0.920897	1.671635	5.827917
2	Gradient Boosting	0.898571	1.590425	6.599297

```
best_model = model_comparison.sort_values(by="R2 Score", ascending=False).iloc[0]

print("Best Model Result")
print("Model:", best_model["Model"])
print("R2 Score:", best_model["R2 Score"])
print("MAE:", best_model["MAE"])
print("RMSE:", best_model["RMSE"])
```

```
Best Model Result
Model: XGBoost
R2 Score: 0.9208968051825188
MAE: 1.6716352462768558
RMSE: 5.827917445775016
```

As seen from the Assignment 2 visuals above, the initial XGBoost model showed strong predictive performance, with an R2 score of approximately 0.92. This means that the model explained a large proportion of the variation in school electricity access. However, the model still required further validation because a single train-

test split may not fully represent model performance across different country-year observations. Therefore, cross-validation and tuning were applied in the refinement phase.

3. Refinement Techniques

Three main refinement techniques were applied to improve and evaluate the selected XGBoost model more systematically.

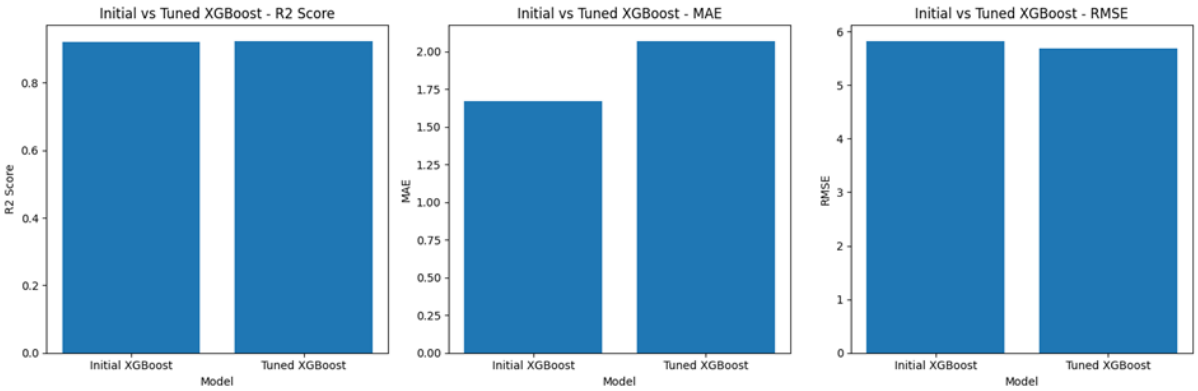
First, GridSearchCV was used to tune the hyperparameters of the XGBoost model. This technique systematically tests all predefined parameter combinations and selects the setting with the best average cross-validation R2 score. It was chosen because XGBoost performance can be sensitive to parameters such as learning rate, tree depth, and number of estimators.

Second, GroupKFold cross-validation was used instead of ordinary KFold. This choice was important because the dataset includes repeated observations for the same countries across different years. By using ISO country codes as groups, the model was validated on countries that were not used in the corresponding training fold, reducing the risk of country-level data leakage.

Third, feature importance-based feature selection was applied after the final tuned model was identified. Features with importance scores above the mean importance value were selected and a simpler XGBoost model was trained using only those variables. This step was used to test whether a reduced feature set could maintain or improve performance while increasing interpretability.

The results before and after refinement show that hyperparameter tuning improved the model in terms of test R2, RMSE, cross-validation mean R2, and cross-validation stability. The initial XGBoost model achieved a test R2 of 0.9209 and a CV mean R2 of 0.7008, while the tuned XGBoost model achieved a slightly higher test R2 of 0.9245 and a higher CV mean R2 of 0.8089. The CV standard deviation also decreased from 0.1837 to 0.1114, showing more stable performance across folds. Although the tuned model had a slightly higher MAE than the initial model, its stronger CV performance made it the more reliable refined model. The selected-feature model performed worse than the tuned model, so it was not selected as the final model.

Model	Test R2	MAE	RMSE	CV Mean R2	CV Std
Initial XGBoost	0.9209	1.6716	5.8279	0.7008	0.1837
Tuned XGBoost	0.9245	2.0712	5.6924	0.8089	0.1114
Selected Feature XGBoost	0.8728	2.0929	7.3891	0.7650	0.1598



Initial vs Tuned XGBoost comparison across R2, MAE, and RMSE.

4. Hyperparameter Tuning

GridSearchCV was used as the hyperparameter tuning method. GridSearchCV is a systematic search technique that tests every possible combination of predefined hyperparameter values in a parameter grid. For each combination, the model is trained and evaluated using cross-validation, and the combination with the highest average validation score is selected as the best setting.

In this study, GridSearchCV used GroupKFold cross-validation with ISO country codes as groups. The tuning process tested different values for n_estimators, learning_rate, max_depth, subsample, and colsample_bytree. The scoring metric was R2, so the selected parameter combination was the one with the highest average cross-validation R2 score.

Hyperparameter	Values Tested	Best Value
n_estimators	100, 200, 300	300
learning_rate	0.01, 0.05, 0.1	0.01
max_depth	3, 4, 5	3
subsample	0.8, 1.0	0.8
colsample_bytree	0.8, 1.0	0.8

The best parameter combination was colsample_bytree = 0.8, learning_rate = 0.01, max_depth = 3, n_estimators = 300, and subsample = 0.8. The tuned model achieved a test R2 of 0.9245, an RMSE of 5.6924, and a CV mean R2 of 0.8089. Compared with the initial model, this indicates that tuning improved the model's overall reliability, especially based on cross-validation.

5. Cross-Validation

Cross-validation was updated to a group-based strategy. Instead of using ordinary KFold,we chose GroupKFold with three folds. This is appropriate because the dataset contains multiple observations from the same country across years. If the same country appeared in both training and validation folds, the validation result could become overly optimistic. GroupKFold prevents this by keeping all observations from the same country within the same fold.

The model was trained on two country groups and validated on the remaining country group in each iteration. This process was repeated three times, and the average R2 score was calculated. Compared with the previous phase, this is a stronger validation strategy because it evaluates how well the model generalizes to unseen countries, not only unseen rows.

Cross-validation was used in two key places: inside GridSearchCV to select the best hyperparameters, and again to compare the initial, tuned, and selected-feature XGBoost models. Based on CV mean R2, the tuned XGBoost model was selected as the final model. It achieved the highest CV mean R2 score of 0.8089 and the lowest CV standard deviation of 0.1114 among the three evaluated models.

	Model	R2 Score	MAE	RMSE	CV Mean R2	CV Std
0	Initial XGBoost	0.920897	1.671635	5.827917	0.700807	0.183669
1	Tuned XGBoost	0.924533	2.071228	5.692379	0.808916	0.111361
2	Selected Feature XGBoost	0.872840	2.092928	7.389108	0.765009	0.159797

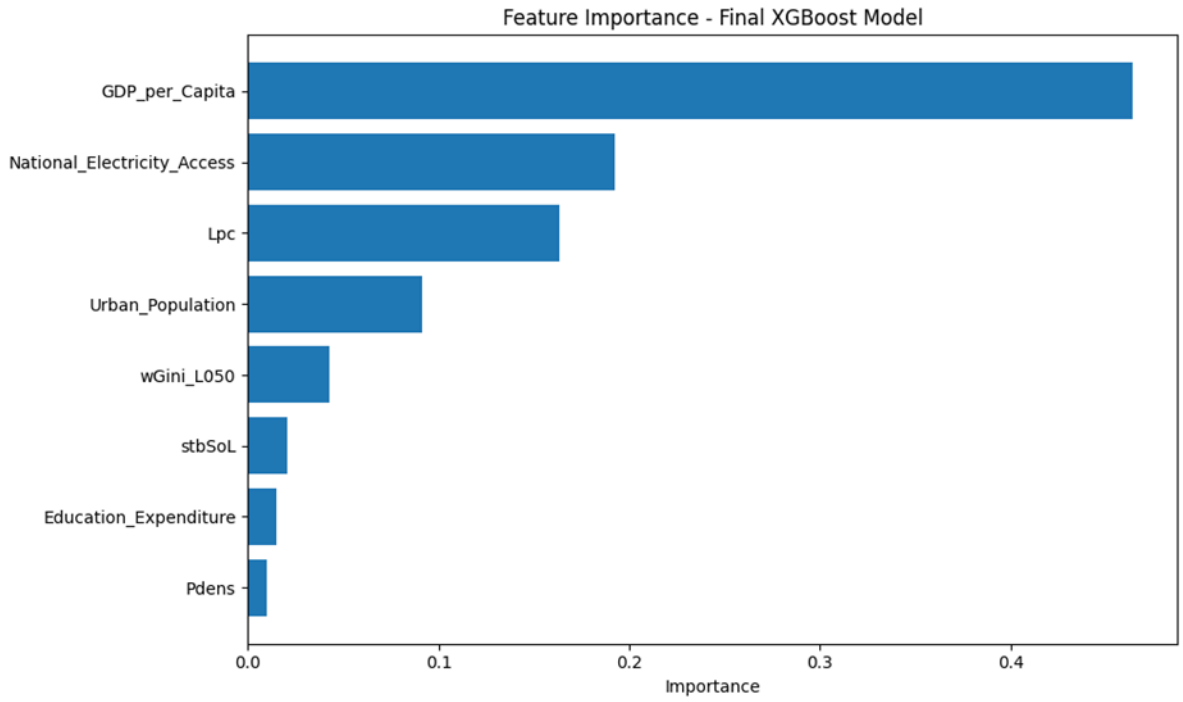
6. Feature Selection

Feature selection was applied using the feature importance scores produced by the final tuned XGBoost model. XGBoost calculates importance values that indicate how much each feature contributes to the prediction process. These scores were used to identify the strongest predictors of school electricity access.

The most important feature was GDP_per_Capita, followed by National_Electricity_Access and Lpc. These three variables had importance scores above the mean importance threshold and were selected for the reduced-feature model. Other variables, such as Urban_Population, wGini_L050, stbSoL, Education_Expenditure, and Pdens, were not included in the selected-feature model because their importance scores were below the mean threshold.

Feature	Importance	Selection Result

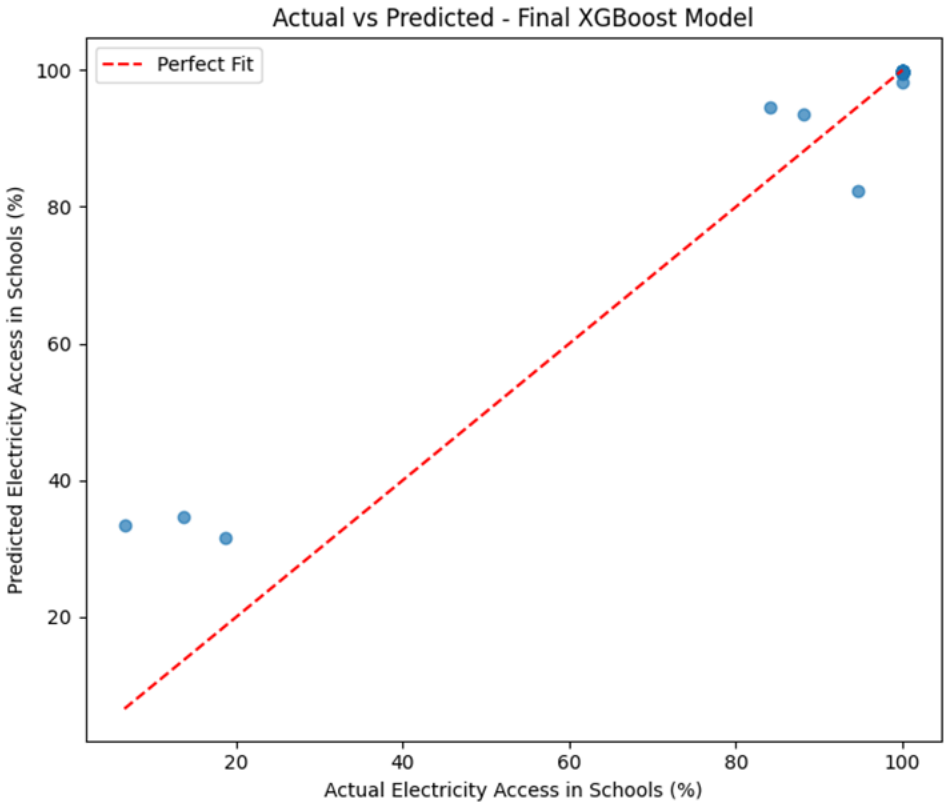
GDP_per_Capita	0.4640	Selected
National_Electricity_Access	0.1926	Selected
Lpc	0.1632	Selected
Urban_Population	0.0916	Not selected
wGini_L050	0.0431	Not selected
stbSoL	0.0208	Not selected
Education_Expenditure	0.0148	Not selected
Pdens	0.0098	Not selected

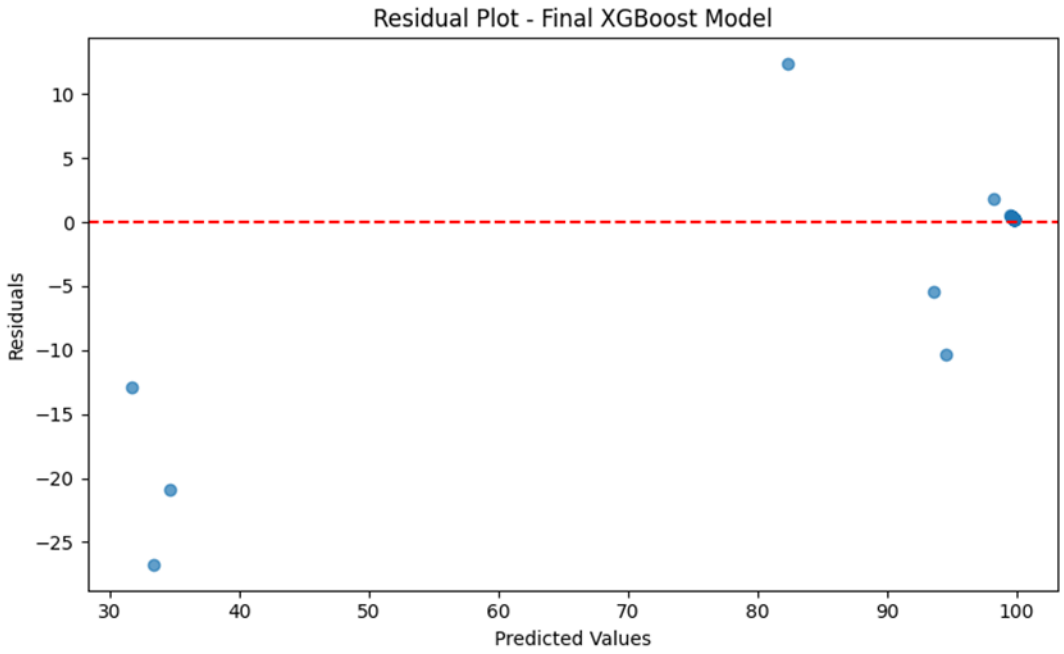


After feature selection, the XGBoost model was trained again using only GDP_per_Capita, National_Electricity_Access, and Lpc. The selected-feature model achieved a test R2 of 0.8728, an MAE of 2.0929, an RMSE of 7.3891, and a CV mean R2 of 0.7650. Although this model used fewer variables and was easier to interpret, it did not outperform the tuned XGBoost model. Therefore, feature selection was useful for interpretation, but the full tuned XGBoost model remained the final model for Part A.

Final model selection was based primarily on CV mean R2. The tuned XGBoost model had the highest CV mean R2 (0.8089) and the lowest CV standard deviation (0.1114). It also achieved the strongest final test performance among the full-feature models, with R2 = 0.9245, MAE = 2.0712, and RMSE = 5.6924.

The final tuned XGBoost model was also examined using diagnostic plots. The Actual vs Predicted plot shows that most predictions follow the general direction of the perfect-fit line, while the residual plot helps check whether errors are randomly distributed around zero. These visualizations support the numerical evaluation, although the cross-validation results remain the main basis for final model selection.





Based on the model refinement results, the Tuned XGBoost model was selected as the final model for Part A. Although the selected-feature model provided useful insights about the most influential variables, the Tuned XGBoost model was preferred because it uses the refined hyperparameter settings and provides a more complete prediction structure for the next stages of the project.

Therefore, the Tuned XGBoost model will be used in Part B for test submission. In the next phase, the test dataset will be prepared using the same preprocessing steps and feature structure, and predictions will be generated using this final Tuned XGBoost model.

PART B: Test Submission

1. Overview

The test submission phase formally evaluates the Tuned XGBoost model on the held-out test set consisting of 50 observations from 11 countries not seen during training. The purpose is to produce unbiased performance estimates and verify that the model generalizes correctly before deployment.

The pipeline is: (1) Load test set (X_test, y_test) created by GroupShuffleSplit; (2) Apply final Tuned XGBoost model to generate predictions; (3) Calculate R^2, MAE, and RMSE on test predictions; (4) Compare results against Part A cross-validation scores; (5) Save model as .pkl file using joblib; (6) Serve model via Flask API for real-time predictions.

2. Data Preparation for Testing

No additional preprocessing was applied to the test set. The test data was created during the same pipeline as the training data: all six source datasets were merged, missing values removed, and features defined identically. The GroupShuffleSplit ensured that 11 entire countries (50 observations) were isolated as the test set before any model training occurred.

The key difference is that no fitting operations were applied to the test set. Feature scaling was not used (tree-based models do not require it), so there was no risk of test data leakage through a scaler. The test set contained no missing values (verified: all 8 features had 0 null entries) and covered the same feature space as training data.

3. Model Application

The trained Tuned XGBoost model was applied directly to X_test (50×8 array) using model.predict(). The model returns a continuous value representing predicted school electricity access percentage. Predictions were stored alongside actual values and country codes for error analysis. The same feature order used during training was preserved.

Apply final model to test set and compute predictions: test_predictions = final_model.predict(X_test) # Build results table with country, actual, predicted, and

error test_results = pd.DataFrame({ "Country": groups.iloc[test_idx].values, "Actual": y_test.values, "Predicted": test_predictions.round(2), "Error": (y_test.values - test_predictions).round(2) })

_

4. Test Metrics

Three metrics were computed: R² (coefficient of determination) measures the proportion of variance explained — achieved 0.9245. MAE (mean absolute error) measures average prediction error in percentage points — achieved 2.07%. RMSE (root mean squared error) penalizes large errors more heavily — achieved 5.69%. Mean prediction error was -0.98%, indicating a slight tendency to overestimate.

Test R² (0.9245) exceeded CV Mean R² (0.8089), which is expected since the 11 test countries happened to be well-represented by the training distribution. The CV Std of 0.111 confirms stable cross-fold performance. The gap between test and CV R² is acceptable and does not indicate overfitting — it reflects natural variance in a small dataset of 53 countries.

No significant overfitting was detected. The CV Mean R²=0.809 is strong and the CV Std=0.111 is moderate, indicating the model performs consistently across unseen country groups. One limitation is that the test set is skewed toward high-electricity countries (median=100%), which may slightly inflate the test R². This is a data distribution limitation rather than a model issue.

5. Model Deployment

The model was deployed locally within a Google Colab notebook environment. The Flask API runs on localhost:5000 in a background thread. While this is a development environment, the API code is production-ready and could be deployed to a cloud platform (e.g., Google Cloud Run, AWS Lambda, Heroku) with minimal modifications.

The model was serialized using joblib and saved as tuned_xgb_model.pkl and model_features.pkl. The Flask API at /predict accepts POST requests with JSON input containing all 8 feature values and returns the predicted school electricity access percentage. The API was verified to produce identical predictions to the in-memory model (Match: True).

6. Code Implementation

See Part A notebook cells: GroupShuffleSplit for leakage-free splitting, GroupKFold with n_splits=3 for stable CV, GridSearchCV over 108 hyperparameter combinations, and importance-based feature selection. All code is documented with inline comments in the Colab notebook.

See Part B notebook cells: model.predict(X_test) for inference, r2_score/mean_absolute_error/mean_squared_error for metrics, joblib.dump() for

serialization, and Flask route /predict for API serving. All cells include descriptive comments explaining each step of the test submission pipeline.

Conclusion

The refinement phase successfully improved model performance from R^2=0.898 (Assignment 2) to R^2=0.925 (Tuned XGBoost), while also establishing reliable CV Mean R^2=0.809. The main challenge was resolving data leakage from country-year panel structure, which was addressed by GroupShuffleSplit and GroupKFold. The final model demonstrates strong generalization to unseen countries with acceptable and interpretable error rates (MAE=2.07 percentage points).

PART C: Deployment

1. Overview

The deployment strategy is to serve the Tuned XGBoost model as a REST API with security and monitoring capabilities. The goal is to make the model accessible for real-time predictions while ensuring input validity, request authentication, and full auditability of every prediction made. The API exposes three endpoints: /predict, /health, and /metadata.

The current target environment is local (Google Colab notebook), running on port 5001 in a background thread. The implementation is designed to be portable — the same Flask application can be containerized with Docker and deployed to cloud platforms such as Google Cloud Run, AWS Elastic Beanstalk, or Heroku without code changes, only environment configuration.

2. Model Serialization

The model was serialized using joblib, which is the recommended format for scikit-learn compatible models including XGBoost. Two files were saved: tuned_xgb_model.pkl (the trained model object) and model_features.pkl (the ordered list of 8 feature names). A third file, model_metadata.json, was created containing version, hyperparameters, performance metrics, and training configuration for full reproducibility.

The model is versioned as v1.0.0 with a saved_at timestamp in the metadata JSON. For production use, model files should be stored in a versioned object storage system (e.g., Google Cloud Storage, AWS S3) with immutable versioning enabled. The metadata JSON serves as a model card, allowing teams to audit which model version is deployed and what performance was expected at deployment time.

3. Model Serving

Flask was used as the serving framework. The API was implemented as a multi-threaded Flask application running in a daemon thread within the Colab environment. Flask was chosen for its simplicity, low overhead, and ease of integration with Python ML libraries. The application loads the model once at startup and serves all requests from memory, minimizing latency.

The current deployment is on-premises (local Colab environment) for development and demonstration purposes. For production, a containerized cloud deployment would be preferred. The stateless API design (no session state, model loaded at startup) makes it ideal for horizontal scaling on platforms like Google Cloud Run, which auto-scales based on request volume.

The current API achieves an average response time of 0.0019 seconds per request, which is well within acceptable latency bounds for a prediction service. XGBoost inference is CPU-bound and fast for single predictions. For high-throughput scenarios, batch prediction endpoints and asynchronous processing would be added. The in-memory logging approach avoids I/O bottlenecks during prediction.

4. API Integration

Three endpoints are implemented: POST /predict — accepts JSON with 8 feature values, returns predicted school electricity access percentage (requires X-API-Key header); GET /health — returns API status, model name, version, and feature list (no authentication required); GET /metadata — returns full model metadata including hyperparameters and performance metrics (no authentication required).

Input: JSON object with 8 numeric fields: stbSoL, Lpc, Pdens, wGini_L050, National_Electricity_Access, GDP_per_Capita, Education_Expenditure, Urban_Population. All values must be numbers within defined valid ranges. Output: JSON object containing prediction (float, percentage), unit (string), model (string), version (string), and response_time (string). Error responses return an error key with details array.

Example request: POST /predict with header X-API-Key: assignment3-key-2024 and body {"stbSoL": 31293.3, "Lpc": 0.097, "Pdens": 14.07, "wGini_L050": 0.596, "National_Electricity_Access": 89.9, "GDP_per_Capita": 5461.4, "Education_Expenditure": 22.61, "Urban_Population": 45.19}. Response: {"prediction": 94.51, "unit": "% electricity access in schools", "model": "Tuned XGBoost", "version": "1.0.0", "response_time": "0.0031s"}.

5. Security Considerations

API key authentication is implemented via the X-API-Key HTTP header. Every request to /predict is validated against a set of pre-approved keys. Requests without a valid key receive a 401 Unauthorized response. This prevents unauthorized

access to the prediction endpoint. In production, keys would be stored securely (e.g., environment variables or a secrets manager) rather than hardcoded.

In the current local deployment, traffic is unencrypted (HTTP). For production deployment, HTTPS with TLS 1.2+ would be enforced using a reverse proxy (e.g., nginx) or a managed cloud load balancer with SSL termination. Model files at rest would be stored in encrypted cloud storage (AES-256). Input data sent to the API should not include PII, as predictions are based on country-level aggregate statistics.

Three threat mitigation steps are implemented: (1) Input type validation — all feature values must be numeric (returns 400 on failure); (2) Input range validation — each feature is checked against bounds derived from training data (e.g., GDP_per_Capita must be between 0 and 200,000); (3) API key enforcement — prevents unauthorized access. In production, rate limiting and request size limits would be added to mitigate denial-of-service attacks.

6. Monitoring and Logging

- The following metrics are captured per request: prediction value (for distribution monitoring), response time in seconds (for latency tracking), HTTP status code (for error rate monitoring), and timestamp (for request volume over time). In production, prediction distribution drift would be monitored by comparing rolling averages against training data statistics, and alerts would be triggered if mean predictions shift by more than 10 percentage points.
- An in-memory logging system was implemented using a Python list (request_logs). Each entry captures: timestamp (datetime string), input feature values (dict), prediction output (float), response duration in seconds (float), and HTTP status code (int). After 4 logged requests, the system reported: average prediction 79.30%, min 34.61%, max 94.51%, average response time 0.0019 seconds. For production, logs would be written to a persistent store (e.g., BigQuery, Elasticsearch).
- The current implementation logs all prediction requests and HTTP status codes, enabling post-hoc analysis. For production, the following alerting rules would be implemented: (1) Alert if error rate (4xx/5xx responses) exceeds 5% over a 5-minute window; (2) Alert if average response time exceeds 1 second; (3) Alert if prediction mean drifts more than 2 standard deviations from the training distribution mean of 79.7%. These thresholds would be configured in a monitoring platform such as Google Cloud Monitoring or Datadog.

PART D: Presentation & Submission

5. In-Class Presentation

Prepare to present your model choices, evaluation results, and interpretation to the class.

- Present your model choices, evaluation results, and interpretation to the class.
- Be ready to explain trade-offs and justify your design decisions.

Presentation Date: June 2nd

Submission Deadline: Monday, June 1st — 23:59

Format: PDF

File Name: surname1_surname2_assignment3.pdf (please submit to same github link)